

Interpolation Search:

If the keys are uniformly distributed between $a[0]$ and $a[n-1]$, this method is more efficient than binary search.

Initially, as in binary search, low is set to 0 and high is set to $n-1$ and the argument key is known to be between $a[\text{low}]$ and $a[\text{high}]$.

On the assumption that the keys are uniformly distributed between these two values, key would be expected to be at approximate position,

$$\text{mid} = \text{low} + (\text{high} - \text{low}) * ((\text{key} - a[\text{low}]) / (a[\text{high}] - a[\text{low}]))$$

If key is less than $a[\text{mid}]$, reset high to $\text{mid}-1$, if key is greater than $a[\text{mid}]$, reset low to $\text{mid}+1$.

This is repeated until the key is found or low becomes greater than high.

The 'C' function is given below:

```
int interpolation_search(int a[], int n, int key)
{
    int low, high, mid;
    low = 0;
    high = n-1;
    while(low<=high)
    {
        mid = low + (high - low) * ((key - a[low]) / (a[high] - a[low]))
        if (key == a[mid]) return mid;
        if (key < a[mid]) high = mid-1;
        else low = mid + 1;
    }
    return(-1);
}
```

